

Moderne Datenbanksysteme und Datenmodelle: Eine praxisorientierte Literaturrecherche

TL;DR

Es gibt kein universelles System. Für OLTP bleibt PostgreSQL die zuverlässigste, am besten validierte Wahl; für OLAP ist ClickHouse (self-hosted) bzw. Snowflake (managed) führend; für Vektorsuche sind pgvector (klein/mittel) und Milvus/Qdrant (groß) die praktisch bewährten Optionen. Die zentrale Empfehlung lautet: spezialisierte, etablierte Systeme kombinieren (Polyglot Persistence) statt ein „Alleskönner“-System zu suchen.

HTAP ist real, aber die Marketing-Zahlen sind mit Vorsicht zu genießen. TiDB berichtet selbst nur $\leq 10\%$ OLTP-Einbruch unter Analytiklast; die unabhängige OLxPBench-Studie misst unter realistischerem Schema bis zu 89% Einbruch. Für die meisten Teams ist ein getrennter OLTP+OLAP-Stack mit CDC (Debezium) robuster und besser verstanden als ein HTAP-System.

Die 5 Systeme, die man kennen muss: PostgreSQL (+ Erweiterung pgvector), ClickHouse, das Lakehouse-Trio (Apache Iceberg/Delta Lake + Spark/Trino), HNSW-basierte Vektorsysteme (Milvus/Qdrant/FAISS) und verteiltes SQL (CockroachDB/Spanner). Für RAG-/AI-Anwendungen ist die Kombination PostgreSQL+pgvector der beste Startpunkt.

Key Findings

1. Operational vs. Analytical ist die fundamentale Achse. OLTP-Systeme (row stores, ACID, niedrige Latenz) und OLAP-Systeme (column stores, vektorisierte Ausführung, data skipping) haben gegensätzliche Architektur-Optimierungen. Ein OLAP-System beantwortet in unter einer Sekunde, wofür ein row-orientiertes OLTP-System Minuten braucht – und umgekehrt.

2. Column Stores + Trennung von Storage und Compute sind die zwei prägenden Architekturinnovationen der letzten 15 Jahre (Snowflake SIGMOD 2016; ClickHouse; BigQuery). Sie ermöglichen elastische Skalierung und günstige Objektspeicherung.

3. Das Lakehouse konvergiert. Apache Iceberg, Delta Lake und Apache Hudi bieten alle ACID, Time Travel und Schema Evolution auf Objektspeicher. Die Unterschiede verschwinden zunehmend (z.B. Delta UniForm, Iceberg v3).

4. HNSW ist der De-facto-Standard für Vektorindizes. Praktisch jede Vektordatenbank (Milvus, Qdrant, Weaviate, Pinecone, pgvector) nutzt HNSW als Default.

5. Verteiltes SQL (NewSQL) – inspiriert von Google Spanner – bringt horizontale Skalierbarkeit + starke Konsistenz zu relationalen Workloads (CockroachDB, YugabyteDB, TiDB).

6. Polyglot Persistence ist der dominante Zustand, nicht die Ausnahme – aber sie erzeugt eine „Steuer“ aus Betriebskomplexität und Synchronisationsproblemen.

Details

1.1 Operational vs. Analytical Databases

Operational Databases / OLTP

PostgreSQL (Kategorie A, etabliert/praxisbewährt) ist das relationale Open-Source-System mit der breitesten Validierung. Row store, MVCC, volle ACID-Konformität, serialisierbare Isolation verfügbar. Es ist durch jahrzehntelangen Produktiveinsatz in Millionen Deployments validiert – die stärkste denkbare Evidenzstufe. Anwendungsfälle: transaktionale Backends, Bestellsysteme, Finanztransaktionen, generell alles mit hoher Schreib-/Lese-Parallelität und niedriger Latenz. Erweiterbarkeit (pgvector, PostGIS, Zeitreihen via TimescaleDB) macht es faktisch zum Multi-Tool. **MySQL** ist die zweite große Option, ebenfalls Kategorie A.

Konsistenz-/Transaktionsmodelle: PostgreSQL bietet die vollständige Palette an SQL-Isolationsleveln (Read Committed default, Repeatable Read über Snapshot Isolation, Serializable über Serializable Snapshot Isolation/SSI).

Analytical Databases / OLAP

ClickHouse (Kategorie A/B) – ursprünglich von Yandex entwickelt, seit Juni 2016 Open Source (Apache 2.0), seit September 2021 als ClickHouse, Inc. kommerzialisiert. Column-oriented DBMS, MergeTree-Storage-Engine, vektorisierte Ausführung. Auf dem ClickBench-Benchmark (43 Queries, 100 GB Hits-Datensatz) ist ClickHouse Cloud das schnellste Angebot: Laut ClickHouse-Bericht „Best columnar databases in 2026“ gilt „ClickHouse Cloud is the fastest service across the 43 queries ... Snowflake is roughly 1.5× slower, BigQuery 2×, Databricks 2.7×, Redshift 5×“. Beim Laden desselben Datensatzes berichtet ClickHouse: „ClickHouse Cloud loads the dataset in 21 seconds versus 2,524 seconds for Snowflake and 1,889 seconds for Redshift Serverless.“ (Wichtig: gehostet vom Hersteller – siehe Caveats.) Schwächen: verteilter Betrieb komplex, begrenzte Indexierung, keine echte Multi-Tenancy, Updates/Deletes schwach. Produktiv u.a. bei Uber, Cloudflare, eBay. Repository: github.com/ClickHouse/ClickHouse. [Flexera + 2](#)

Snowflake (Kategorie A) – Dageville, Cruanes, Zukowski et al., „The Snowflake Elastic Data Warehouse“, SIGMOD 2016, DOI 10.1145/2882903.2903741. Prägende Innovation: multi-cluster, shared-data-Architektur mit drei Schichten (Cloud Services, Virtual Warehouses = isolierte Compute-Cluster, Data Storage auf S3). Trennung von Storage und Compute; Micro-Partition-Pruning statt Indizes; VARIANT-Typ für semi-strukturierte Daten mit Schema-on-Read (nur ~10% Overhead); Time Travel und Zero-Copy Cloning via MVCC auf immutablen Dateien. Vollständig managed (SaaS). Schwäche: Kosten, Vendor Lock-in.

Apache Druid (Kategorie A/B) – Sub-Sekunden-OLAP auf hochdimensionalen/high-cardinality Daten, modulare Microservice-Architektur, Segmente. Stärke: Echtzeit-Ingestion + niedrige Latenz. Schwäche: viele bewegliche Teile, begrenzte Indexierung. Verwandt: Apache Pinot (single-digit-ms p99 durch Star-Tree-/Inverted-Indizes). [db-engines](#)

[Flexera](#)

Die **Lakehouse-Systeme** (Databricks/Delta Lake, Iceberg) werden in Abschnitt 1.3 behandelt.

HTAP

Die zentrale HTAP-Referenz ist der Survey von Chao Zhang, Guoliang Li, Jintao Zhang et al., „HTAP Databases: A Survey“, IEEE TKDE, April 2024, arXiv:2404.15670. Er klassifiziert tightly-coupled HTAP-Architekturen in vier Kategorien (u.a. Primary Row Store + In-Memory Column Store; Distributed Row Store + Column Store Replica) und identifiziert den zentralen Trade-off: **Data Freshness vs. Performance Isolation**.

TiDB (Kategorie B) – Huang, Liu, Cui, Fang et al. (PingCAP), „TiDB: A Raft-based HTAP Database“, PVLDB 13(12):3072-3084, 2020, DOI 10.14778/3415478.3415535. Erste industrielle Real-time-HTAP-Publikation. Architektur: Multi-Raft-Storage mit Row Store (TiKV, Rust, RocksDB) + Column Store (TiFlash), verbunden über asynchrone Raft-Log-Replikation zu „Learner“-Knoten. Stateless SQL-Layer (Go). MySQL-kompatibel. Open Source (github.com/pingcap/tidb). [ACM Digital Library + 3](#)

Evidenzbewertung HTAP – der kritische Punkt dieses Themenfelds:

Herstellerangabe (selbst evaluiert): Im VLDB-Paper (Abschnitt 6.4, CH-benCHmark, 100 Warehouses) berichtet PingCAP: „more analytical processing clients degrade the TP throughput at most 10% compared to no AP clients“ und OLAP „with at most only a 5% drop“. Zum Vergleich zeigten sie, dass MemSQL 7.0 unter Analytiklast „dropping by more than five times“ erleidet.

Unabhängige Widerlegung: OLxPBench (Kang, Wang, Gao, Tang, Zhan, ICDE 2022, DOI 10.1109/ICDE53745.2022.00182, arXiv:2203.16095) argumentiert, das CH-benCHmark-„Stitch“-Schema verschleierte die Interferenz, weil viele Analytik-Queries nie aktualisierte Tabellen lesen. Unter semantisch konsistentem Schema messen die Autoren für TiDB bis zu **89% OLTP-Einbruch und 59% OLAP-Einbruch**: „the throughput interference of OLTP and OLAP is as high as 89% and 59%, respectively“ und explizit: „the analytical workloads decrease transactional throughput by 89% using the semantically consistent schema, rather than 10% using stitch schema in the previous work“. Eine Echtzeit-Query in einer Hybrid-Transaktion verschlechtert Latenz und Durchsatz um Faktor ~5,9.

Unabhängiger Head-to-Head-Vergleich: TiQuE (Faria, Pereira, Alonso, Vilaça et al., INESC TEC/U. Minho + MonetDB, PVLDB 16(9):2274-2288, 2023, DOI 10.14778/3598581.3598598): Auf TPC-C erreicht TiDB 6.5.0 **450 tx/s** vs. SingleStore 8.0.4 **331 tx/s**; auf CH-benCHmark ist TiDB (**188 s** Gesamtantwortzeit) schneller als SingleStore (**236 s**). Beide werden jedoch von klassischem PostgreSQL (**1.415 tx/s** im reinen OLTP) deutlich übertroffen. Beide HTAP-Systeme „slowly decreasing operational throughput with each new analytical background worker“ – d.h. Isolation ist gut, aber nicht perfekt.

SingleStore (Kategorie B, ehemals MemSQL) – Prout et al., „Cloud-Native Transactions and Analytics in SingleStore“, SIGMOD 2022. In-Memory Row Store + On-Disk Column Store in einer Engine, kompilierte Query-Ausführung, Trennung von Storage und Compute. Kommerziell, nur Read Committed.

Weitere HTAP-relevante Systeme: PolarDB, CockroachDB (eher verteiltes OLTP), Google AlloyDB, Snowflake Unistore, MariaDB ColumnStore.

Praktische Bewertung HTAP: HTAP funktioniert, aber die Isolationsversprechen sind stark schema- und workload-abhängig. Für die meisten Teams ist ein getrennter Stack (OLTP + CDC + OLAP) besser verstanden, reproduzierbarer und robuster als ein einzelnes HTAP-System.

1.2 Datenbanken nach Datenstruktur

Tabular / Relational

Row vs. Column Stores: Row stores (PostgreSQL, MySQL) für OLTP; Column stores (ClickHouse, Snowflake) für OLAP. Query Optimization über kostenbasierte Optimizer, B-Tree-Indizes (OLTP) vs. data skipping / Zonemaps / Min-Max-Statistiken (OLAP).

Verteiltes SQL (NewSQL):

Google Spanner (Kategorie A) – Corbett et al., „Spanner: Google’s Globally-Distributed Database“, OSDI 2012. Prägend: die **TrueTime-API** (Atomuhren + GPS-Empfänger in jedem Rechenzentrum) für global geordnete Transaktionen mit „external consistency“. Laut dem Spanner-Paper erzeugt TrueTime „time uncertainty intervals of between 1 and 7 milliseconds“; das Unsicherheitsfenster ϵ liegt bei Googles Infrastruktur typischerweise unter 4 ms und erzeugt einen entsprechenden Commit-Wait. Proprietär, Google Cloud. Beeinflusste die gesamte NewSQL-Kategorie.

CockroachDB (Kategorie B) – von Ex-Google-Ingenieuren 2015 gegründet, inspiriert von Spanner. Verteilter Key-Value-Store (RocksDB) + SQL-Layer, PostgreSQL-kompatibel, Raft-basiert, multi-active availability (alle Knoten akzeptieren Reads/Writes). Open Source (Core). Nutzt Raft statt TrueTime.

[medium](#)

↗
[Conduktor](#)

↗
YugabyteDB (Kategorie B) – Spanner-Architektur + wiederverwendeter PostgreSQL-Query-Layer (YSQL) auf verteiltem DocDB-Storage (inspiriert von HBase), Raft für Leader-Election und Replikation. Open Source.

Trade-off verteiltes SQL: horizontale Skalierbarkeit + Geo-Verteilung + starke Konsistenz, aber höhere Latenz und Betriebskomplexität als Single-Node PostgreSQL. TiDB nutzt das Percolator-Modell (globaler Timestamp-Oracle), was Multi-Region-OLTP wegen WAN-Latenz erschwert; Spanner/CockroachDB/YugabyteDB skalieren Multi-Region besser.

Graph Databases

Zwei Modelle: **Property Graphs** (Knoten/Kanten/Properties, Query via Cypher/Gremlin) vs. **RDF Graphs** (Triples Subjekt-Prädikat-Objekt, Query via SPARQL).

Neo4j (Kategorie A) – bekannteste Graph-DB mit der größten Community, native Graph-Storage, Query-Sprache **Cypher** (deklarativ, „pattern matching“, leicht lernbar – „it reads like a pattern: find this node, follow this relationship, return the connected entity“). Voll ACID. Cloud (Aura), on-prem oder hybrid. Reiches Ökosystem (APOC-Prozeduren, Bloom, Browser). Cypher wird via openCypher/GQL standardisiert. [Brilworks](#)

↗
[Analytics Vidhya](#)

↗
Amazon Neptune (Kategorie A) – General Availability am 30. Mai 2018 (laut AWS-Pressemitteilung: „announced general availability of Amazon Neptune, a fast, reliable, and fully managed graph database service“; Preview am 29. Nov. 2017 angekündigt). Fully managed AWS-Service, unterstützt beide Modelle: Property Graph (Gremlin, openCypher) und RDF (SPARQL). Immediate consistency auf dem Writer,

eventual auf Replicas. Multi-AZ-Replikation, automatisches Failover. Schwäche: single label pro Knoten, keine APOC-Prozeduren, einige Cypher-Klauseln fehlen, kleinere Community. [Cycode](#)

↗

Apache Jena (RDF/SPARQL, Java-Framework) und **GraphDB** (Ontotext, RDF) für semantische/Ontologie-Anwendungen und Wissensgraphen.

Query-Sprachen: **Cypher** (deklarativ, SQL-nah, produktivste für Application-Entwickler), **Gremlin** (imperativ, Traversal-orientiert, Apache TinkerPop, mächtig aber schwerer wartbar), **SPARQL** (RDF-Standard, W3C, exzellent für Ontologien).

Anwendungsgebiete: Betrugserkennung, Empfehlungssysteme, Wissensgraphen, Netzwerk-/Abhängigkeitsanalyse, Identity Graphs. Vorteil gegenüber relational: Multi-Hop-Traversals ohne teure rekursive JOINS.

Vector Databases

Grundlage: Speicherung hochdimensionaler Embeddings (typisch 768 oder 1536 Dimensionen aus ML-Modellen), Suche via **Approximate Nearest Neighbor (ANN)** – ein Tausch von Genauigkeit (Recall) gegen Geschwindigkeit. Zentraler Survey: Pan, Wang, Li, „Survey of vector database management systems“, The VLDB Journal 33:1591-1615, 2024, DOI 10.1007/s00778-024-00864-x (arXiv:2310.14021): „There are now over 20 commercial vector database management systems (VDBMSs), all produced within the past five years.“

[Springer](#)

↗

Index-Algorithmen (Funktionsweise):

HNSW (Hierarchical Navigable Small World) – Malkov & Yashunin, arXiv:1603.09320 (2016), IEEE TPAMI 42(4):824-836, 2018/2020. Mehrschichtiger Proximity-Graph: obere Layer grob (wenige Knoten, weite Sprünge – ähnlich Skip Lists), untere Layer fein (alle Knoten). Suche startet oben, geht „greedy“ zum nächsten Nachbarn und dann Layer für Layer nach unten. Logarithmische Suchkomplexität. Parameter: **M** (Verbindungen pro Knoten), **ef_construction** (Bauqualität), **ef_search** (Suchtiefe). Stärke: beste Latenz/Recall-Balance, inkrementelle Inserts ohne Rebuild. Schwäche: hoher Speicherbedarf (Graph 2-5× der Rohdaten), langsamerer Build. [arXiv](#)

↗

[Medium](#)

↗

IVF (Inverted File) – k-means-Clustering des Vektorraums in Zellen; Suche nur in den nächstgelegenen Zellen. Speichereffizient (~1× Rohdaten), aber schlechtere Cluster-Qualität in hohen Dimensionen (Fluch der Dimensionalität), braucht periodisches Re-Clustering bei wachsendem Korpus. Varianten: IVF_FLAT, IVF_PQ (Product Quantization für Kompression), IVF_SQ8.

DiskANN – disk-basierte Graphen für sehr große Datensätze bei begrenztem RAM.

Systeme:

FAISS (Kategorie A) – Meta/Facebook AI Research; Bibliothek, kein vollständiges DBMS. Johnson, Douze, Jégou, „Billion-scale similarity search with GPUs“, arXiv:1702.08734, IEEE TBD 7(3):535-547, 2019. CPU + GPU, alle Index-Typen (Flat, IVF, HNSW, PQ). GPU-Implementierung 8,5× schneller als vorheriger GPU-State-of-the-Art. Referenzimplementierung, in viele Systeme (Weaviate, Qdrant historisch) eingebettet. Repository: [github.com/facebookresearch/faiss](#). [SciSpace](#)

↗

Milvus (Kategorie B) – Wang et al., „Milvus: A Purpose-Built Vector Data Management System“, SIGMOD 2021, S.2614-2627, DOI 10.1145/3448016.3457550 (271+ Zitationen). Cloud-native, disaggregierte Storage/Compute-Microservices (Query-, Data-, Index-Nodes getrennt skalierbar), reichste Index-Auswahl (HNSW, IVF-Varianten, DiskANN), Tech-Stack etcd + MinIO/S3 + Pulsar/Kafka. Für Milliarden-Vektoren. Schwäche: hohe Betriebskomplexität. Repository: github.com/milvus-io/milvus. [ACM Digital Library + 3](#)

↗
Qdrant (Kategorie B) – in Rust geschrieben, HNSW, exzellentes Metadata-Filtering (ACORN-Algorithmus 2025 für gefilterte HNSW), starke Single-Node-Performance + Raft-basierter Cluster-Modus. Beste Performance-pro-Dollar für Self-Hosting; laut mehreren Vergleichen führend bei konsistent niedriger Latenz.

Weaviate (Kategorie B) – Go, HNSW, native Hybrid Search (BlockMax WAND + Relative Score Fusion), Vektorisierungs-Module, GraphQL-API. [Onideli](#)

↗
pgvector (Kategorie A/B) – PostgreSQL-Erweiterung, HNSW (ab v0.5.0) + IVFFlat. Aktuelle Versionen unterstützen halfvec, sparsevec und binäre Quantisierung; v0.8.x verbesserte das Filtering (Query-Planner-Integration). Für RAG-Workloads bis ~50 Mio. Vektoren produktionsreif. Vorteil: Vektoren + relationale Daten in EINEM System, in einer Transaktion, kein separater Stack. Repository: github.com/pgvector/pgvector.

Unabhängige Benchmarks: ann-benchmarks (Aumüller, Bernhardsson, Faithfull, SISAP 2017 / Information Systems 87, 2020, DOI 10.1016/j.is.2019.02.006) ist der etablierte neutrale Vergleichsstandard für ANN-Algorithmen (fixe Hardware, automatische Parametersuche, Recall vs. QPS). Neuer: VIBE (Vector Index Benchmark for Embeddings, arXiv:2505.17810). Hersteller-Benchmarks (Qdrant, Milvus) sind mit Vorsicht zu lesen.

Metadata Filtering (WHERE-Klauseln kombiniert mit Vektorsuche) ist ein zentrales Praxisproblem – naive Postgres-Query-Planner wählen manchmal sequentielle Scans statt des ANN-Index.

Qdrant/Weaviate/Milvus haben spezialisierte gefilterte HNSW-Traversierung; Milvus 2.5+, Qdrant v1.9+ und Weaviate unterstützen native Hybrid Search (dense + sparse BM25).

1.3 Zusammenspiel verschiedener Datenbanktypen

Polyglot Persistence (Begriff geprägt von Martin Fowler in seinem bliki-Beitrag „PolyglotPersistence“ vom 16. Nov. 2011: „we are gearing up for a shift to polyglot persistence where any decent sized enterprise will have a variety of different data storage technologies for different kinds of data“). Ein E-Commerce-Backend braucht legitim: relationale DB (Bestellungen), Document Store (Produktinhalte), Key-Value-Cache (Sessions), Suchindex (Katalog), Graph (Empfehlungen). Industriedaten (DB-Engines, CNCF-Survey) zeigen: Multi-Database-Deployments sind der Normalfall, nicht die Ausnahme. [PuppyGraph](#)

↗
Trade-off spezialisiert vs. Multi-Model: Polyglot = beste Einzeltools, aber Betriebskomplexität, mehrfache Datenkopien, Synchronisationsprobleme, und Fehlertoleranz = schwächste Komponente im Stack (um eine globale Query zu beantworten, müssen alle Stores verfügbar sein). Multi-Model-DBs (ArangoDB, Oracle, PostgreSQL mit Erweiterungen) reduzieren die Betriebssteuer, sind aber oft nicht best-of-breed pro Modell. Empirische Studien zeigen gemischte Ergebnisse: Polyglot kann Multi-Model bei Multi-Thread-Queries übertreffen, während Multi-Model bei Zuverlässigkeit/Verfügbarkeit punktet.

Synchronisation/Konsistenz: Zwischen separaten Stores gibt es keine ACID-Garantien; Konsistenz ist ein „permanentes low-grade Problem“. Lösungen: CDC, Outbox-Pattern (Debezium Outbox Event Router), idempotente Consumer, Dead-Letter-Queues. [PuppyGraph](#)

Change Data Capture (CDC): Erfasst INSERT/UPDATE/DELETE aus dem Transaktionslog (WAL/binlog) und streamt sie in Echtzeit. **Debezium** (Open Source, log-basiert, JVM, meist mit Kafka Connect) ist der De-facto-Standard. Typische Pipeline: Quelldatenbank → Debezium → Kafka/Redpanda → Sinks (ClickHouse als read-optimierter Sink, Elasticsearch, Iceberg-Lakehouse, Warehouse). Vorteile log-basierter CDC: minimaler Quell-Impact, erfasst Deletes und alte Record-States, keine Schemaänderung nötig. Best Practices: Schema-Registry von Anfang an, Replication-Lag überwachen, exactly-once bzw. at-least-once mit idempotenten Consumern. Alternativen: AWS DMS, Google Datastream, Oracle GoldenGate (kommerziell), Flink CDC. [Substack + 3](#)

Data Warehouse vs. Data Lake vs. Lakehouse:

Data Warehouse: strukturierte Daten, Schema-on-Write, teuer, hohe Performance (Snowflake, BigQuery, Redshift).

Data Lake: Rohdaten auf günstigem Objektspeicher, Schema-on-Read, flexibel, aber ohne ACID-Garantien und Governance.

Lakehouse: kombiniert beides – Warehouse-Garantien (ACID, Time Travel, Schema Evolution) auf günstigem Objektspeicher. Zentrale Table Formats:

Delta Lake – Armbrust et al., „Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores“, PVLDB 13(12):3411-3424, 2020. Von Databricks; stärkste Performance mit Spark/Photon.

Transaction Log + Checkpoints; passt zu Spark/Databricks-zentrierten Teams und Echtzeit-Pipelines.

Apache Iceberg – von Netflix entwickelt, an die Apache Software Foundation gespendet. Beste Schema Evolution (field ID tracking), hidden partitioning, Engine-unabhängig (Portabilität über Spark, Trino, Snowflake, Dremio) → verhindert Vendor Lock-in. Ideal für große, read-heavy Analytik über viele Engines.

Apache Hudi – bietet zusätzlich Storage-Engine-Funktionalität (Data Lakehouse Management System), stark bei update-heavy CDC-Workloads.

Konvergenz: Alle drei bieten äquivalente ACID-Garantien; Delta UniForm (Delta lesbar als Iceberg) und Iceberg v3 verwischen die Grenzen. Performance-Unterschiede stammen primär von der Query-Engine, nicht vom Format. Benchmarks: LST-Bench (Camacho-Rodríguez et al., SIGMOD/PACMMOD 2024).

Rolle in AI/RAG-Systemen: Retrieval-Augmented Generation (RAG; Lewis et al., NeurIPS 2020) kombiniert einen Retriever mit einem generativen LLM, um Halluzinationen zu reduzieren und aktuelles/domänenspezifisches Wissen einzubinden. Der Retriever nutzt eine Vektordatenbank für semantische Suche über Embeddings. Zentrale Surveys: Ranjan et al., „A Comprehensive Survey of RAG“, arXiv:2410.12837 (2024); Gao et al.; sowie Graph-RAG-Surveys (arXiv:2408.08921). RAG war 2024 das dominierende Thema der NLP-Systemforschung. Architektur-Muster: Dokumente chunken → embedden → in Vektor-DB → Query embedden → ANN-Suche (Top-k) → Kontext an LLM. Erweiterungen: Hybrid Search (dense + sparse BM25) für Fachterminologie/Eigennamen, Reranking (Cross-Encoder), Graph-RAG für Multi-Hop-Reasoning, CRAG/RAG-Fusion. Kernaussage der Praxis: Chunking-Strategie und Embedding-Qualität sind wichtiger als die Wahl der konkreten Vektor-DB. [arXiv](#)

Empirischer Vergleich (Kernkriterien)

System	Kategorie	Qualität/Evidenz	Geschwindigkeit	Skalierbarkeit	Implementierungsaufwand	Verbreitung
PostgreSQL	A	Höchste (Jahrzehnte Produktiveinsatz)	OLTP top	vertikal + Read-Replicas	niedrig	auf fast allen Plattformen
ClickHouse	A/B	Hoch (ClickBench, breite Produktion)	OLAP top	horizontal (komplex)	mittel	auf fast allen Plattformen
Snowflake	A	Hoch (SIGMOD 2016, Massenmarkt)	OLAP sehr gut	elastisch	niedrig (managed)	auf fast allen Plattformen
TiDB	B	Gemischt (Hersteller ≤10% vs. unabh. bis 89% Interferenz)	mittel	horizontal, Multi-Raft	hoch	gibt es nur bei Alibaba
CockroachDB	B	Gut (Spanner-Design)	mittel (Geo-Latenz)	horizontal, Multi-Region	mittel-hoch	gibt es nur bei Google
Neo4j	A	Hoch (Marktführer Graph)	Graph-Traversal top	mittel	niedrig-mittel	auf fast allen Plattformen
FAISS	A	Hoch (Referenz, Milliarden-Skala)	sehr schnell	Bibliothek, kein Cluster	mittel (kein DBMS)	z.B. bei Facebook, Spotify
Milvus	B	Hoch (SIGMOD 2021, Produktion)	schnell, Bulk-Indexing top	Milliarden-Vektoren	hoch	gibt es nur bei Alibaba
Qdrant	B	Gut (Rust, unabh. Benchmarks)	niedrigste Latenz	single-node stark + Cluster	niedrig-mittel	auf fast allen Plattformen
pgvector	A/B	Gut (PostgreSQL-Basis)	gut bis ~50M Vektoren	via PostgreSQL	sehr niedrig	auf fast allen Plattformen

Präsentations-Storyline

Problem: Datenvielfalt (transaktional, analytisch, graph, vektoriell) + gegensätzliche Optimierungsziele → kein System kann alles.

Bisherige Lösung: Ein monolithisches RDBMS für alles → skaliert nicht; OLAP-Queries blockieren OLTP.

Aktuelle funktionierende Systeme: OLTP (PostgreSQL), OLAP (ClickHouse/Snowflake), Lakehouse (Iceberg/Delta), Graph (Neo4j), Vektor (Milvus/Qdrant/pgvector), verteiltes SQL (CockroachDB), HTAP (TiDB).

Technische Unterschiede: Row vs. Column Store; Storage/Compute-Trennung; HNSW vs. IVF; Raft/TrueTime-Konsens; Open Table Formats.

Empirischer Vergleich: ClickBench (OLAP), ann-benchmarks (Vektor), OLxPBench/TiQuE (HTAP), TPC-C/TPC-H/CH-benCHmark (OLTP/OLAP/HTAP).

Praktische Bewertung: Herstellerbenchmarks kritisch prüfen (TiDB 10% vs. 89% ist DAS Lehrbeispiel!); auf unabhängige, reproduzierbare Studien setzen.

Empfehlung: Polyglot mit etablierten Systemen + CDC zur Synchronisation; einfach starten, spät spezialisieren.

Recommendations

Die 5 Systeme, die man wirklich kennen muss:

PostgreSQL (+ pgvector, + weitere Erweiterungen) – das zuverlässigste, vielseitigste Fundament. Beginne hier für fast jeden Anwendungsfall.

ClickHouse (self-hosted) oder **Snowflake** (managed) – für analytische Workloads.

Lakehouse (Apache Iceberg oder Delta Lake + Spark/Trino) – für großskalige, engine-unabhängige Datenplattformen.

HNSW-basierte Vektorsuche (Milvus für Milliarden-Skala, Qdrant für mittlere Größe, pgvector für den Einstieg; FAISS als Referenzbibliothek) – für RAG/AI.

Verteiltes SQL (CockroachDB/YugabyteDB, oder Spanner managed) – wenn Geo-Verteilung + starke Konsistenz nötig ist.

Staged (stufenweise) Empfehlungen mit Schwellwerten:

Stufe 1 (Start/Prototyp): PostgreSQL + pgvector. Deckt OLTP, moderate Analytik und Vektorsuche in EINEM System ab. Geringster Aufwand, höchste Reproduzierbarkeit. Wechsle erst weg, wenn eine konkrete Grenze erreicht ist.

Stufe 2 (Analytik wächst): Füge ein OLAP-System (ClickHouse) hinzu, synchronisiert via Debezium-CDC.

Schwellwert: analytische Queries verlangsamen den OLTP-Betrieb oder dauern regelmäßig > mehrere Sekunden.

Stufe 3 (Vektor skaliert): Wechsle von pgvector zu Qdrant/Milvus. **Schwellwert:** > ~50 Mio. Vektoren, oder dedizierte Filter-/Hybrid-Search-Anforderungen, oder pgvector-HNSW-Speicher/Latenz wird zum Engpass.

Stufe 4 (Geo/Konsistenz): CockroachDB/YugabyteDB. **Schwellwert:** Multi-Region-Deployment + starke Konsistenz zwingend.

Stufe 5 (Data Platform): Lakehouse (Iceberg). **Schwellwert:** mehrere Engines/Teams arbeiten auf denselben Daten; Data-Lake-Governance wird nötig.

Welches funktioniert am zuverlässigsten? PostgreSQL – unangefochten durch Evidenz (Jahrzehnte Produktiveinsatz) und Reifegrad. **Am besten zum Selbst-Implementieren/Lernen?** pgvector + HNSW; der HNSW-Algorithmus (Malkov & Yashunin) ist exzellent dokumentiert und FAISS bietet eine lesbare Referenzimplementierung zum Nachvollziehen. Eine eigene HNSW-Implementierung ist als Lernprojekt machbar; für die Produktion sollte man bestehende Bibliotheken (hnswlib, FAISS) übernehmen. **Am**

besten für den State-of-the-Art-Praxiseinsatz? ClickHouse (OLAP), Qdrant (Vektor mittlere Größe), Snowflake/Databricks (managed Platform). **Was man zunächst ignorieren kann:** Multi-Model-„Alleskönner“ mit dünner Historie; rein experimentelle ANN-Varianten und learned indexes (Kategorie C/D); HTAP-Systeme, sofern kein zwingender Bedarf an Echtzeit-Analytik direkt auf Live-Transaktionsdaten besteht – ein CDC-getrennter Stack ist meist robuster, reproduzierbarer und besser verstanden.

Caveats

Herstellerbenchmarks sind systematisch optimistisch. Das TiDB-Beispiel ($\leq 10\%$ selbst berichtet vs. bis 89% unabhängig via OLxPBench gemessen) zeigt exemplarisch, wie stark Benchmark-Schema und -Wahl Ergebnisse verzerren. Vertraue unabhängigen, reproduzierbaren Studien (ann-benchmarks, ClickBench mit fixer Hardware, OLxPBench, TiQuE). Beide TiDB-Zahlen sind reale Messungen – unter unterschiedlichen, umstrittenen Schemata.

ClickBench wird von ClickHouse, Inc. gehostet; obwohl methodisch offen (fixe c6a.4xlarge-Hardware für Open-Source-Engines, öffentlich einreichbar), besteht ein Interessenkonflikt. Ähnlich sind Qdrant- und PingCAP-Benchmarks parteiisch.

Einzelknoten-Benchmarks verteilter Systeme (wie in TiQuE, das TiDB/SingleStore auf einer 32-vCPU-Cloud-Instanz testet) benachteiligen Systeme, die primär für Multi-Node-Deployments ausgelegt sind – die Autoren weisen selbst darauf hin.

„2026“-datierte Quellen in dieser Recherche sind teils Blog-/Marketing-Inhalte, keine Peer-Review; für Fakten wurden Primärquellen (SIGMOD/VLDB/TPAMI/ICDE/OSDI) priorisiert.

Das Vektor-DB-Feld bewegt sich sehr schnell – konkrete QPS/Recall-Zahlen veralten innerhalb von Monaten. Führe eigene Benchmarks auf eigenen Daten und mit dem eigenen Embedding-Modell durch.

Kategorien C/D (rein experimentelle/theoretische Ansätze wie neue ANN-Graph-Varianten, learned indexes, unvalidierte Multi-Model-Systeme) wurden bewusst nur am Rande behandelt, da praktische, unabhängige Validierung fehlt – gemäß dem Fokus auf nachweislich funktionierende, einsetzbare Systeme.